

LAIRS Audit Fabric Standard

Beyond Blockchain: Deterministic, Segmented, Cryptographically Anchored Logging for Governed Systems with Distributed Verification

Author: Russell Aydelotte · Aegis Secure Convergence Systems · February 18, 2026

Version 2

Modern infrastructure lacks mechanisms to establish provable continuity of system behavior under adversarial or partial-failure conditions. Traditional logging systems rely on implicit trust in storage layers, while blockchain-based approaches impose excessive consensus overhead and lack integration with governed runtime environments. This paper introduces the LAIRS Audit Fabric Standard, a deterministic, cryptographically anchored audit architecture combining governance-aware events, segmented Merkle sealing, and bounded lifecycle storage. The architecture is extended through SentiNet, a distributed verification fabric that propagates trust state, policy bundles, and integrity assertions across nodes without reliance on consensus-based ledgers.

■ Platform Status Notice

LAIRS is an active research and architectural exploration platform. It is **not production ready** and is intended for controlled experimentation. Components marked *In Development* are under active work and subject to change. Do not deploy in production environments without explicit authorization.

1. Introduction

As computing systems become autonomous and interconnected, the integrity of operational history becomes essential for trust, safety, and accountability. The LAIRS ecosystem introduces a distributed, governance-aware audit and verification fabric capable of preserving provable system truth without distributed consensus. This standard defines the architecture, components, and compliance requirements for systems seeking to implement the LAIRS Audit Fabric.

2. Problem Statement

Current audit mechanisms fail to simultaneously satisfy the requirements of modern governed infrastructure. The following critical properties are absent from existing solutions:

- Tamper-evident continuity of operational history
- Governance-aware context embedded in audit records
- Bounded storage lifecycle with preserved cryptographic integrity
- Offline and independent verification capability
- Distributed trust propagation across heterogeneous nodes
- Fail-closed integrity guarantees under partial failure

Blockchain systems address decentralized trust but introduce consensus complexity, unbounded storage growth, and semantic mismatch unsuitable for infrastructure telemetry and runtime governance.

3. Architectural Overview

The LAIRS Audit Fabric is composed of four integrated planes operating in a strict hierarchy of trust and enforcement:

Plane	Component	Role
Enforcement Plane	Sentinel	Runtime policy enforcement and event generation
Governance Plane	Metatron's Gate	Authority validation, artifact issuance, quorum decisions
Audit Fabric	Ledger + Merkle	Cryptographic event anchoring and segment sealing
Verification Fabric	SentiNet	Distributed trust propagation and node integrity

4. System Pipeline

Events flow through a deterministic pipeline from runtime origin to verified storage:



5. Segmented Merkle Ledger Model

The LAIRS Audit Fabric uses segmented Merkle ledgers rather than monolithic append-only logs. Events are grouped into discrete segments that undergo a deterministic sealing process, producing portable, independently verifiable trust artifacts:

- Merkle tree computation over segment events
- Root generation producing a canonical segment fingerprint
- Payload hashing binding event content to the segment seal
- Digital signing by the governance authority
- Ledger commitment creating an immutable provenance anchor

This model enables partial verification of historical segments without requiring access to the full audit history, supporting bounded storage and offline validation.

6. Governance-Coupled Event Semantics

Audit events in the LAIRS Fabric are not passive log entries. Each event carries embedded governance context, transforming logs into decision provenance records. Required event fields include:

Field	Purpose
Actor Identity	Cryptographically verified identity of the acting principal
Role Classification	Authority classification at time of action
Intent Metadata	Declared purpose of the action for governance traceability
Policy Constraints	Active policy envelope governing the action
Approval State	Quorum or unilateral approval status with signatories
System Context	Runtime state snapshot at time of event creation

7. Distributed Verification Fabric — SentiNet

SentiNet functions as the distributed trust propagation layer of the LAIRS Audit Fabric. Rather than relying on consensus-based ledgers, SentiNet propagates cryptographically signed state across nodes using a push-verify model with local enforcement capability:

- Signed policy bundles distributed to enforcement nodes
- Integrity verification against known-good Merkle roots
- Last Known Good fallback on verification failure
- Fail-closed enforcement when trust state cannot be confirmed
- Cross-node trust propagation without central coordination
- Independent audit segment verification at each node

8. Storage Lifecycle Model

The LAIRS Audit Fabric introduces bounded lifecycle storage across three tiers, each preserving cryptographic integrity guarantees regardless of storage medium:

Tier	Characteristics	Verification
Hot	Active segments, immediate access, full event resolution	Real-time Merkle and hash chain
Warm	Sealed segments, compressed storage, recent history	Segment root verification
Cold	Archived artifacts, offline media, long-term retention	Offline signature and root check

9. Verification and Replay

Independent validation is a first-class capability of the LAIRS Audit Fabric. Any authorized party can verify the integrity of historical records without access to the originating system. Verification comprises four deterministic steps:

- Segment signature verification against the issuing authority's public key
- Merkle root recomputation from raw event payloads
- Hash chain continuity check across segment boundaries
- Governance metadata validation confirming policy and approval state

Replay capability allows historical scenarios to be re-executed against current policy, supporting regression testing, incident investigation, and compliance auditing.

10. Security Analysis

The LAIRS Audit Fabric architecture provides cryptographic mitigations against a comprehensive threat model:

Threat	Mitigation
--------	------------

Tampering	Hash chain continuity and Merkle root binding detect any modification
Insider manipulation	Quorum approval requirements and signed artifacts limit unilateral action
Storage corruption	Segment-level Merkle sealing isolates and localizes corruption
Ledger truncation	Chain continuity checks detect gaps; SentiNet propagates last-good state
Replay attacks	Time-bounded tokens and segment sequence numbers prevent replay
Node compromise	Fail-closed enforcement and Last Known Good fallback contain impact
Policy corruption	Signed policy bundles with independent verification detect tampering

11. Comparison With Blockchain

Dimension	Blockchain	LAIRS Audit Fabric
Trust model	Consensus-based	Cryptographic + governance-native
Storage growth	Unbounded	Bounded lifecycle tiers
Verification	Requires network access	Fully offline capable
Governance	External / implicit	Embedded in every event
Latency	Consensus-dependent	Deterministic, local
Recovery	Fork resolution	Last Known Good + fail-closed
Semantics	Transaction-oriented	Decision provenance records

12. Implementation Evidence

The LAIRS Audit Fabric Standard is not theoretical. The architecture has been implemented with the following active capabilities verified in deployment:

- Active governance events with embedded policy and approval metadata
- Merkle-sealed segments with independently verifiable root artifacts
- Signed audit artifacts distributed across SentiNet nodes
- Distributed policy verification without central coordination
- Runtime enforcement integration with Sentinel agent plane

13. Applications

The LAIRS Audit Fabric Standard is applicable across high-assurance environments where provable operational continuity and governed accountability are required:

- Critical infrastructure protection and operational continuity
- Industrial control systems requiring deterministic audit trails
- Defense networks with distributed verification requirements
- Autonomous AI governance with provable decision provenance
- Sovereign cloud environments requiring offline verification capability
- Supply chain verification with cryptographic continuity guarantees

14. Conclusion

The LAIRS Audit Fabric Standard demonstrates that distributed governance assurance can be achieved without consensus-based ledgers. Through deterministic cryptographic design, governance-coupled event semantics, and the SentiNet distributed verification fabric, the architecture establishes a new paradigm for provable system integrity.

By combining segmented Merkle sealing, bounded lifecycle storage, and fail-closed enforcement, the LAIRS Audit Fabric delivers tamper-evident continuity, offline verification, and governance-native auditability — properties that blockchain systems cannot simultaneously provide for infrastructure telemetry at scale.

Russell Aydelotte

Aegis Secure Convergence Systems · February 18, 2026

Appendices

Appendix A — Two-Node Deployment Topology

The current implementation operates across two physical systems connected through the SentiNet verification channel. Each node fulfills a distinct role in the governance and verification architecture:

GL360 Node — Primary

Produces authoritative audit artifacts.

Lenovo Node — Secondary

Provides verification separation from origin environment.

Appendix B — Ledger Event Structure Example

A representative ledger event includes the following fields:

```
{
  "ts": 1739634561.123,
  "event_type": "SEGMENT_SEALED",
  "hash": "sha256:...",
  "prev_hash": "sha256:...",
  "payload_sha256": "sha256:...",
  "merkle_root": "sha256:...",
  "segment_basename": "segment_YYYYMMDD_timestamp.jsonl.gz",
  "sig_basename": "segment_YYYYMMDD_timestamp.jsonl.gz.sig"
}
```

Key field properties:

- **hash** — Links the ledger entry into a hash chain
- **prev_hash** — Provides continuity verification across entries
- **merkle_root** — Commits the contents of the segment
- **payload_sha256** — Protects the compressed artifact
- **sig_basename** — Points to signature file enabling independent authenticity validation

Appendix C — Segment Artifact Lifecycle

Each audit segment follows a deterministic lifecycle from event generation to archived storage:

1. Events generated by governance engine
2. Events written to segment file
3. Merkle tree computed
4. Root hash generated
5. Segment payload hashed
6. Digital signature applied
7. Ledger entry recorded

8. Artifact stored in audit archive

Segment artifacts consist of two files per segment:

```
segment_xxx.jsonl.gz  
segment_xxx.jsonl.gz.sig
```

Segments remain independently verifiable regardless of storage location.

Appendix D — Verification Procedure Example

Independent verification can be performed using the following steps:

1. Decompress segment artifact
2. Recompute Merkle tree from event records
3. Compare computed root to ledger root
4. Validate digital signature
5. Confirm hash chain continuity across ledger entries

Verification does not require access to the original runtime system.

Appendix E — Governance Identity Controls

The governance identity layer implements authenticated attribution for all governance decisions. Current capabilities include:

- Password hashing using scrypt
- Time-based one-time password (TOTP) MFA
- Enrollment validation workflows
- Policy-controlled authentication
- Identity event logging

Identity data is associated with governance decisions within audit records. The system provides authenticated attribution but does not claim legal non-repudiation guarantees.

Appendix F — SentiNet Verification Channel Behavior

The SentiNet channel currently supports:

- Signed artifact transfer between nodes

- Signature verification
- Known-good state referencing
- Validation independent of origin node

The channel does not currently implement:

- Multi-peer consensus
- Automatic mesh propagation
- Byzantine fault tolerance

Its purpose is deterministic verification within a governed trust boundary.

Appendix G — Storage Layout Example

Representative storage structure:

```
/mnt/sentinet_logs/  
lairs_audit_archive/  
segment_*.jsonl.gz  
segment_*.jsonl.gz.sig  
merkle_ledger.jsonl
```

- Hot storage for active segments
- Warm storage for historical data
- Cold storage for archival retention

Integrity guarantees are preserved through cryptographic artifacts rather than storage medium trust.

Appendix H — Security Assumptions

The current implementation operates under the following security assumptions:

- Cryptographic primitives are not compromised
- Signing keys are protected
- Nodes operate within a governed administrative boundary
- Physical compromise of both nodes simultaneously is unlikely

These assumptions are typical for enterprise infrastructure security models.

Appendix I — Limitations of Current Implementation

The following limitations are acknowledged to avoid overstating system maturity. Each represents a known constraint of the current two-node deployment:

- Two-node deployment only
- Enforcement agent in observe mode
- No hardware root-of-trust integration (see planned integration below)
- Limited automated distribution
- No formal verification proofs

Planned Hardware Root of Trust Integration

Future revisions of the LAIRS Audit Fabric will integrate TPM-backed key sealing and measured boot validation to bind governance signing authority to hardware state. This will enable silicon-anchored artifact issuance and boot integrity attestation without altering the deterministic verification model described herein.

This catalog reflects the current public architecture as of February 18, 2026. Components are classified as **Implemented** (operational in the current two-node deployment) or **In Development** (under active design or integration). The Human Master Administrator retains ultimate authority over all components at all times.

✓ **Implemented — Operational in current deployment**

● **In Development — Under active design or integration**

Implemented Components

LAIRS Core

Role: Cognitive runtime and orchestration layer

✓ **Implemented — Operational**

- › Runtime coordination and event routing between all modules
- › Identity, memory, and state management
- › Policy hooks, audit logging, and explainability
- › Local-first execution — no external dependency for core operation
- › No autonomous irreversible actions without human approval

SentiNet Fabric

Role: Zero-trust secure networking and governance fabric

✓ Implemented — Operational

- › Zone-based segmentation with default-deny communication model
- › Internal DNS and PKI anchoring
- › Controlled bridge modes between trust zones
- › Every module treated as untrusted until verified
- › Policy enforcement occurs at the fabric layer
- › Last Known Good (LKG) state validation

Metatron's Gate

Role: Quorum governance and system authority control

✓ Implemented — Operational

- › Explicit intent declaration required for all sensitive actions
- › Multi-party quorum enforcement: Machine / AI / Human
- › Cryptographic approval and denial tracking
- › Observe-only and enforcement modes
- › Prevents unilateral system modification by design
- › Append-only audit trail with causal chaining

Sentinel Agent

Role: Host-level integrity, telemetry, and local attestation

✓ Implemented — Operational

- › Host-level monitoring and integrity checks
- › Secure telemetry to LAIRS Core
- › Foundation module for trust establishment
- › Currently operating in observe mode

AiCraft

Role: Local AI training and lifecycle governance

✓ Implemented — Functional for controlled training

- › Dataset ingestion and validation
- › Model training and evaluation
- › Versioned model artifacts
- › Transparency into training behavior
- › Explicit datasets only — no hidden learning
- › Human-directed experimentation

In Development

Sentinel Gate

Role: Policy enforcement and controlled access (Gated Community)

● In Development

- › Entry control point into SentiNet fabric
- › Policy validation at ingress
- › Trust scoring and rejection handling
- › Fail-closed by design

Sentinel AV

Role: Malware detection and quarantine

● In Development

- › Local scanning without cloud dependency
- › Quarantine workflows
- › Integration with Sentinel Agent and Sentinel Gate

Sentinel Forge

Role: Human-approved governed system evolution

● In Development

- › Detects issues and performance degradation
- › Generates candidate patches for human review
- › Stages changes with full rollback capability
- › Requires cryptographic human approval before application
- › Changes applied only at reboot boundaries
- › Advisory only — cannot self-modify autonomously

Leave No Trace (LNT)

Role: Data minimization and forensic hygiene

● In Development

- › Log scrubbing and data retention control
- › Ephemeral runtime handling
- › Privacy-preserving operations
- › Minimizes data footprint without compromising audit integrity

Guiding Philosophy: Digital Sovereignty through Local Intelligence, Governed Evolution, and Zero Trust by Default. LAIRS is designed to empower operators, not replace them. Intelligence is local. Authority remains human. Evolution is deliberate.

Appendix J — Terminology

Audit Fabric

Cryptographically sealed logging system with deterministic verification.

Governance Engine

Component responsible for policy evaluation and decision recording.

Verification Channel

Mechanism used to distribute and validate signed state across nodes.

Segment

Bounded group of events sealed using a Merkle tree and signature.

Known-Good State (LKG)

Previously validated configuration used for fallback during verification failure.